Installation

The toolchain this pipeline needs, and why the shortcuts fail silently

Install this before starting the agent, not after

`Multi_Agent_Outline.md` opens by probing the toolchain, and it will tell you if something is missing. But every failure mode below is **silent** — the build succeeds, produces a PDF that looks fine, and passes a naive check, while the accessibility tagging is quietly absent. Fixing that after the agent has already transcribed a document means redoing review rounds that already closed. Install everything on this page first.

The one command

```
bash Installation/install_toolchain.sh          # TeX Live 2024+, ~1.2 GB, 15-30 min
export PATH=$HOME/texlive/2026/bin/x86_64-linux:$PATH
pip install -r Installation/requirements.txt    # pymupdf, pillow, numpy
sudo apt install poppler-utils                  # pdftoppm -- the reference renderer
```

Then install **veraPDF** (free, https://verapdf.org) — the conformance checker Step 4d runs against every compiled PDF.

That is the whole install. The rest of this document explains *why* two of these steps cannot be swapped for the obvious alternative already on most machines.

Why TeX Live must be 2024 or later

The TeX Live shipped by Debian/Ubuntu's own package repositories is several years behind. It matters more than a normal version lag: **that older kernel cannot tag mathematics at all.** It accepts the `\DocumentMetadata{...}` declaration this pipeline's documents open with, silently ignores the accessibility `testphase` modules named inside it, and compiles a PDF that *looks* identical to a properly tagged one — same text, same layout — but carries no structure tree worth mentioning. Feed that PDF to a conformance checker and it still passes, because an untagged document that happens to contain readable text can clear a shallow check. The defect only shows up once someone actually walks the tag tree, which is exactly what this pipeline's own Step 4d tests do and a quick glance does not.

`install_toolchain.sh` installs a current upstream TeX Live into `$HOME/texlive` rather than touching the system installation, specifically so it does not fight with `apt` or anything else that depends on the system's LaTeX.

Verify it, don't assume it. After installing, compile the included probe:

```
lualatex Installation/probe.tex
lualatex Installation/probe.tex
lualatex Installation/probe.tex   # three passes -- MathML needs the third
```

Then open `probe.pdf` and confirm it actually contains `/Formula`, `/L`, `/Table`, `/TH`, and `/Alt` in its tag structure (a PDF text editor or `pdftotext -raw` plus a look at the file with a hex/text viewer will show

these). If any is absent, the toolchain is not ready — do not start the agent yet.

Why the page renderer is `pdftoppm`, not PyMuPDF

Every seam row and figure crop box the pipeline records during segmentation is a **pixel coordinate in one specific raster**. Poppler's `pdftoppm` and PyMuPDF's own rendering disagree — different antialiasing, and on US Letter at 300 dpi, a different page height outright (3301 rows against 3300). `render_pages.py` in `tools/` prefers `pdftoppm` and falls back to PyMuPDF only if poppler is unavailable, recording which engine ran either way — but a set of crop boxes measured under one engine does not transfer to a page rendered under the other. Install `poppler-utils` so the fallback is never needed.

The rest of the list

- **`pymupdf`, `pillow`, `numpy`** (via `pip install -r Installation/requirements.txt`) — used throughout `tools/` for image processing, PDF structure inspection, and the numeric accessibility tests in Step 4d.
- **veraPDF** — the PDF/UA conformance checker. Free, scriptable, no account needed. This pipeline treats a conformance pass as a floor, not evidence (see `Accessibility_Info.pdf`), but it is still a mandatory first check every compiled document goes through.
- **PAC 2026** (optional, Windows-only) — the institutional checker an accessibility reviewer at many universities will actually run. It has no command-line interface, so it cannot be scripted into this pipeline the way veraPDF is, but running it manually against a finished PDF and folding the result back in (as this project's own `PIPELINE_LOG.md` documents doing, more than once) is worth the manual step if a Windows machine is reachable. Free from axes4 GmbH: `winget install axes4.PAC`.

Confirm everything together

Once installed, `Reproduce.pdf` walks through the verification commands that prove this whole toolchain — not just LaTeX — is working correctly against a known-good example before you point it at real material.